

Sprint 10 Structured Notes: Industry Git Expansion and CI/CD

1. Current Progress

You are now at **Sprint 10** of the Relational Databases learning path.

The full learning path is:

Tools → Terminal → Bash → Bash Scripting → Git → PostgreSQL → SQL → Normalization, Security, Performance

So far, you have already finished the Git foundation:

Sprint 5: Nano and Git Setup

You learned:

- what Git is
- what version control means
- what a commit is
- what a branch is
- Git vs GitHub
- Git installation and configuration

Sprint 6: Local Git Workflow

You learned:

- local repository
- hidden `.git` folder
- working tree
- untracked, modified, staged
- staging area
- `git status`
- `git add`
- `git reset` for unstaging
- `git commit`
- `git log`

Sprint 7: GitHub Repository and Remotes

You learned:

- local vs remote repository
- GitHub repository
- `origin`
- `git remote -v`
- `git clone`
- `git push`
- `git pull`
- HTTPS vs SSH

Sprint 8: Branches, Pull Requests, and Code Review

You learned:

- branches
- feature branches
- pull requests
- base branch and compare branch
- diff view on GitHub
- code review
- merge, squash merge, rebase merge

Sprint 9: Collaboration, Forks, Sync, .gitignore, Pages, and Conflicts

You learned:

- forking
- write access
- `origin` vs `upstream`
- contributor workflow
- syncing with upstream
- `.gitignore`
- GitHub Pages
- merge conflicts
- conflict markers
- rebase basics
- interactive rebase
- squash

Now: Sprint 10

Sprint 10 is the last Git sprint before PostgreSQL and SQL.

Simple meaning:

Sprint 1 to Sprint 9 taught you the main Git workflow. Sprint 10 teaches you the extra Git skills that real teams expect developers to know.

2. Main Goal of Sprint 10

The main goal of Sprint 10 is:

Fill the beginner Git gaps that teams expect.

By the end of Sprint 10, you should understand:

- how to inspect changes more carefully
- how to undo work safely
- how to save unfinished work temporarily
- how to fix the last commit
- how to recover after mistakes
- how to mark release points
- how to copy one commit from one branch to another
- how to understand remote branch references
- how common team workflows work
- how CI/CD works at a beginner level
- how GitHub Actions workflow files are organized

Simple meaning:

Sprint 10 is about safer Git work, cleaner history, and basic automation.

3. Why Sprint 10 Is Important

Earlier Git sprints taught you the normal workflow:

```
edit → stage → commit → push → pull request → merge
```

Sprint 10 improves that workflow.

It teaches you how to:

- check changes before committing
- undo the correct thing in the correct way
- avoid rewriting shared history by mistake
- recover from mistakes
- work more like real development teams
- understand the automation that checks code in modern projects

Simple meaning:

Sprint 10 helps you avoid common Git mistakes and understand the professional workflow around Git.

4. Big Beginner Idea

Until now, your Git workflow looked like this:

```
git status
git add file
git commit -m "message"
git push
```

Sprint 10 adds extra safety around it:

```
git status
git diff
git add file
git diff --staged
git commit -m "message"
git push
```

Simple meaning:

Do not just save changes. First check exactly what changed.

This is one of the most useful professional Git habits.

Part 1: Checking Changes Carefully

5. What Is `git diff`?

`git diff` shows the changes in your working tree that are **not staged yet**.

Command:

```
git diff
```

It shows things like:

- added lines
- removed lines
- changed lines

Simple meaning:

`git diff` **shows what changed before you stage it.**

Example

You edit `README.md`. Before running `git add README.md`, run:

```
git diff
```

Now Git shows exactly what changed in the file.

Why It Matters

Use `git diff` when:

- you want to check your edits
- you are not sure what changed
- you want to avoid staging accidental edits

Good Beginner Habit

```
git status  
git diff  
git add README.md
```

6. What Is `git diff --staged`?

After you stage changes using `git add`, you can inspect the staged version using:

```
git diff --staged
```

Simple meaning:

`git diff --staged` **shows what is ready for the next commit.**

Difference Between the Two

Command	What it shows
<code>git diff</code>	changed but not staged yet
<code>git diff --staged</code>	already staged for commit

Why This Matters

Sometimes you stage the wrong file or too many files. `git diff --staged` helps you review the next commit before creating it.

Good Workflow

```
git diff
git add README.md
git diff --staged
git commit -m "docs: update README"
```

Part 2: Restore and Unstage

7. What Is `git restore <file>`?

`git restore <file>` removes local changes from a file and returns it to the last committed version.

Example:

```
git restore README.md
```

Simple meaning:

Throw away my local changes in this file.

Important Warning

If you use `git restore README.md`, your current local edits in that file are removed.

Use it only when:

- you are sure you do not want the edits
- the changes were only local

- you want to start that file again from the last commit
-

8. What Is `git restore --staged <file>?`

This removes a file from the staging area, but keeps the file changes in your working tree.

Example:

```
git restore --staged README.md
```

Simple meaning:

Unstage the file, but keep my edits.

Important Idea

This is the modern beginner-friendly version of unstaging. Earlier you learned:

```
git reset README.md
```

Now you should understand:

- `git reset README.md` can unstage
 - `git restore --staged README.md` is more specific and clearer for this job
-

9. `git rm <file>`

`git rm <file>` removes a file and stages that removal for the next commit.

Example:

```
git rm notes.txt
```

Simple meaning:

Delete this file and prepare that deletion for commit.

Why Use It?

You could delete a file normally, but `git rm` is useful because Git understands that the file removal is part of your next commit.

10. `git mv oldname newname`

`git mv` renames or moves a file in a Git-aware way.

Example:

```
git mv oldname.md newname.md
```

Simple meaning:

Rename or move this file and prepare that change for commit.

Why Use It?

You could use normal `mv`, but `git mv` makes the intention clearer inside Git work.

Part 3: Stash

11. What Is `git stash`?

Sometimes you are working, but you are not ready to commit yet. Then suddenly you need to:

- switch branches
- pull updates
- fix another urgent task

That is where stash helps.

Command:

```
git stash
```

Simple meaning:

Temporarily save my unfinished work and hide it for now.

It does **not** make a normal commit. It stores the unfinished changes in a temporary Git storage area.

12. Useful Stash Commands

```
git stash
```

Temporarily save unfinished work.

```
git stash list
```

Show saved stashes.

```
git stash list
```

Simple meaning:

Show the temporary work bundles I saved earlier.

```
git stash pop
```

Bring the latest stash back and remove it from the stash list.

```
git stash pop
```

Simple meaning:

Bring my hidden work back and remove that stash entry.

```
git stash apply
```

Bring a stash back, but keep the stash entry.

```
git stash apply
```

Simple meaning:

Bring the saved work back, but keep the stash copy too.

```
git stash drop
```

Delete a stash entry.

```
git stash drop
```

Simple meaning:

Remove a saved stash entry I no longer need.

13. When to Use Stash

Use stash when:

- your work is not ready to commit
- you must switch tasks quickly
- you need to move to another branch
- you want to pull updates first

Example Flow

```
git status
git stash
git checkout main
git pull
git checkout feature
git stash pop
```

Simple meaning:

Hide unfinished work → do another task → bring the work back later.

Part 4: Amend, Revert, Reset, and Recovery

14. What Is `git commit --amend`?

`git commit --amend` lets you change the most recent commit.

Example:

```
git add missing-file.md
git commit --amend
```

Simple meaning:

Fix my last commit instead of making a new separate one.

Common Uses

Use amend when:

- you forgot to add one file to the last commit
- you want to improve the last commit message
- the last commit is still local and not shared yet

Important Warning

Use amend mainly before pushing.

Simple rule:

Amend local unpushed commits. Be careful with already shared commits.

15. What Is `git revert <commit>`?

`git revert` safely undoes a commit by creating a new commit that cancels the old one.

Example:

```
git revert abc123
```

Simple meaning:

Make a new commit that undoes an old commit.

Why Revert Is Safe

Revert does not erase history. It keeps the bad commit in history and adds a new undo commit.

Example Idea

Old history:

```
A -- B -- C
```

After revert of `C`:

```
A -- B -- C -- D
```

Where `D` cancels `C`.

Best Use

Use revert when:

- the commit is already pushed
- the commit may already be shared
- you want a safe public undo

16. What Is `git reset`?

`git reset` moves Git history backward. It is more powerful and more dangerous than revert.

Common forms:

```
git reset --soft HEAD~1
git reset --mixed HEAD~1
git reset --hard HEAD~1
```

Meanings

Command	Simple meaning
<code>git reset --soft HEAD~1</code>	undo last commit, keep changes staged
<code>git reset --mixed HEAD~1</code>	undo last commit, keep changes unstaged
<code>git reset --hard HEAD~1</code>	undo last commit and delete changes

Important Warning

`git reset --hard` can destroy local work.

Simple meaning:

Reset changes branch history. Hard reset can remove work completely.

17. Revert vs Reset

This is one of the most important Sprint 10 ideas.

Command	Best use	Safety
<code>git revert</code>	undo shared/public commit	safer
<code>git reset</code>	fix local/private history	more dangerous

Simple Rule

- If the commit is already pushed or shared, prefer `git revert`
- If the commit is only local and private, `git reset` may be okay

Simple meaning:

Revert adds a safe undo commit. Reset changes history itself.

18. What Is `git reflog`?

`git reflog` shows where your branch head has been.

Command:

```
git reflog
```

Simple meaning:

Git's recovery history.

Why It Matters

Use reflog when:

- you reset wrongly
- you moved branches wrongly
- you lost track of a commit
- you want to recover an old branch position

Important Idea

- `git log` shows normal project history
- `git reflog` helps recover after mistakes

Simple meaning:

Reflog is a recovery tool.

19. What Is `git show <commit>`?

`git show <commit>` shows details of one specific commit.

Example:

```
git show abc123
```

Simple meaning:

Show me this commit and its exact changes.

Why It Matters

It helps you inspect one commit carefully instead of scrolling through all history.

20. What Is `git blame <file>`?

`git blame <file>` shows which commit and which author last changed each line in a file.

Example:

```
git blame README.md
```

Simple meaning:

Show who last changed each line in this file.

Why It Matters

Useful for:

- understanding old code or text
 - finding when a line changed
 - tracing the last change for a specific line
-

Part 5: Tags and Release Points

21. What Is a Tag?

A tag marks an important point in Git history.

Example:

```
git tag v1.0.0
```

Simple meaning:

Mark this commit as version 1.0.0.

Common Uses of Tags

- release version
 - stable point
 - project milestone
-

22. Lightweight Tag vs Annotated Tag

Lightweight Tag

```
git tag v1.0.0
```

Simple meaning:

A simple label on a commit.

Annotated Tag

```
git tag -a v1.0.0 -m "release v1.0.0"
```

Simple meaning:

A richer tag with a message and extra release information.

Push a Tag

```
git push origin v1.0.0
```

Simple meaning:

Send the tag to GitHub.

Part 6: Cherry-Pick and Detached HEAD

23. What Is `git cherry-pick <commit>`?

Cherry-pick copies one selected commit onto your current branch.

Example:

```
git cherry-pick abc123
```

Simple meaning:

Take this one commit and apply it here.

When It Is Useful

Use it when:

- one small fix exists on another branch
- you need only one commit, not the whole branch
- you want to copy one focused change

Beginner Warning

Cherry-pick is useful, but do not overuse it. Too much cherry-picking can make history harder to understand.

24. What Is Detached HEAD?

Detached HEAD means you are looking directly at a commit instead of working on a normal branch.

Example:


```
git checkout abc123
```

Simple meaning:

I am standing on a commit, not on a branch.

Danger

If you make commits in detached HEAD and do not create a branch, that work can become hard to find.

Safe Action

```
git switch -c save-my-work
```

Simple meaning:

Create a branch if you want to keep work from detached HEAD.

25. What Is `git switch -` ?

`git switch -` returns to the previous branch.

Example:

```
git switch -
```

Simple meaning:

Go back to the branch I was using before.

This is useful when moving quickly between two branches.

Part 7: Remote Branches and Related Commands

26. What Are Remote Branches?

A remote branch is a branch that exists on a remote repository like GitHub.

Examples:

- `origin/main`
- `origin/feature`

Simple meaning:

Remote branches are GitHub-side branch references that your local Git knows about.

27. `git branch -r`

This command shows remote branches.

```
git branch -r
```

Simple meaning:

Show remote branch names.

28. `git fetch --all`

This downloads updates from all configured remotes.

```
git fetch --all
```

Simple meaning:

Update my knowledge of all remotes.

It does not automatically merge changes into your current branch.

29. `git pull --rebase`

This brings remote changes and then reapplies your local commits on top.

```
git pull --rebase
```

Simple meaning:

Download updates, then place my local commits on top of the newer remote work.

Why It Exists

It helps keep history more straight.

Beginner Warning

Use it carefully after you understand rebase better.

30. `git remote rename origin upstream`

This renames a remote.

Example:

```
git remote rename origin upstream
```

Simple meaning:

Change the nickname of this remote.

This is useful in more advanced remote setups, but beginners mainly need to know that remote names can be changed.

Part 8: Cleaning and Safe Inspection

31. `git clean -n`

`git clean -n` shows what untracked files would be removed.

```
git clean -n
```

Simple meaning:

Preview what would be deleted.

The `-n` means dry run or preview.

32. `git clean -fd`

This removes untracked files and folders.

```
git clean -fd
```

Simple meaning:

Delete untracked files and folders.

Important Warning

This can remove files you still wanted. Always prefer preview first:

```
git clean -n
```

Part 9: Branch Protection and Team Workflows

33. What Is Branch Protection?

Branch protection means adding rules to protect important branches such as `main`.

Common Branch Protection Rules

- pull request required before merge
- approval required
- CI checks must pass
- direct push to `main` blocked
- required reviews before merge

Simple meaning:

Branch protection prevents people from breaking the stable main branch accidentally.

This is very common in team projects.

34. What Is GitHub Flow?

GitHub Flow is a simple workflow used by many teams.

Basic GitHub Flow

`main` → create short branch → commit → push → pull request → review → merge into `main`

Simple meaning:

Short branch, review it, then merge it into main.

Why It Is Good for Beginners

GitHub Flow is usually the easiest team workflow to start with. It keeps the process simple.

35. What Is Trunk-Based Development?

Trunk-based development means developers merge small changes into a stable main branch often.

Simple meaning:

Keep branches short and merge frequently into a stable main branch.

Why It Works

It works best when teams have:

- strong tests
 - reliable CI
 - small focused changes
 - feature flags when needed
-

36. What Is Git Flow?

Git Flow is a more structured branching model.

It may use branches such as:

- `main`
- `develop`
- `feature/...`
- `release/...`
- `hotfix/...`

Simple meaning:

Git Flow is a bigger branch system for projects with release cycles.

Beginner Rule

Know Git Flow exists, but do not start with it first. For most beginners, GitHub Flow is easier.

Part 10: CI/CD and GitHub Actions

37. What Is CI?

CI means **Continuous Integration**.

Simple meaning:

CI automatically checks your code when changes are pushed or when pull requests are opened.

CI can do things like:

- install dependencies
 - run lint checks
 - build the project
 - run tests
-

38. What Is CD?

CD means **Continuous Delivery** or **Continuous Deployment**.

Simple meaning:

CD helps prepare or deploy accepted changes automatically.

Easy Difference

- CI checks work automatically
 - CD helps deliver or deploy work automatically
-

39. What Is GitHub Actions?

GitHub Actions is GitHub's automation system.

It can run workflows when:

- you push code
- you open a pull request
- changes are merged

Simple meaning:

GitHub Actions is the tool GitHub uses to run automatic checks and automation steps.

40. What Is a Workflow File?

A workflow file is usually a YAML file. It tells GitHub Actions what to do.

Workflow files are stored inside:

```
.github/workflows
```

Example:

```
.github/workflows/checks.yml
```

Simple meaning:

GitHub looks inside `.github/workflows` to find automation instructions.

41. What Is YAML Here?

YAML is a text format often used for configuration. In Sprint 10, it is used to describe GitHub Actions workflows.

Simple meaning:

YAML is the format used to write workflow instructions.

42. Important GitHub Actions Workflow Parts

A workflow can define these parts:

name

Workflow name.

Simple meaning:

What this workflow is called.

on

The trigger.

Simple meaning:

When this workflow should run.

Examples:

- on push
- on pull request

jobs

Groups of work.

Simple meaning:

The main work sections inside the workflow.

runs-on

The runner operating system.

Simple meaning:

Which machine GitHub should use to run the job.

Example:

- ubuntu-latest

steps

Small actions inside a job.

Simple meaning:

The ordered list of things the job will do.

uses

Use an existing action.

Simple meaning:

Use a ready-made GitHub Action from somewhere else.

with

Pass settings or inputs to an action.

Simple meaning:

Give extra values to the action being used.

run

Run a command.

Simple meaning:

Run a shell command in the workflow.

43. Common GitHub Actions Example Commands

The course mentions example automation steps like:

```
pnpm install
pnpm run lint
pnpm run build
pnpm run test
```

Simple meaning:

These are examples of steps that CI may run automatically to check that the project still works.

44. Example Beginner Workflow

```
name: Simple Check

on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main

jobs:
  check:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Print message
        run: echo "GitHub Actions is working"
```

Simple Meaning of This Example

- the workflow is called `Simple Check`
- it runs on pushes to `main`
- it also runs on pull requests to `main`
- it has one job called `check`
- GitHub uses an Ubuntu runner
- it checks out the code
- then it runs one command

Part 11: Extra Team Topics from the Industry Expansion List

45. What Is `.gitattributes`?

`.gitattributes` controls special file behavior in Git. One common use is managing line endings across Windows, macOS, and Linux.

Simple meaning:

`.gitattributes` helps Git handle files more consistently across different operating systems.

46. Conflict Edge Cases

Some merge conflicts are harder than simple text conflicts. Examples include:

- binary file conflicts
- deleted vs modified file conflicts
- renamed file conflicts
- whitespace-only conflicts
- line ending differences

Simple meaning:

Not all conflicts are simple line-vs-line text conflicts. Some conflict cases need extra care.

Part 12: Sprint 10 Practice Flow

Use a practice repository for Sprint 10 work. Do not test dangerous commands on important projects.

47. Practice A: Check Changes Before Commit

```
git status
echo "Sprint 10 diff practice" >> README.md
git diff
git add README.md
git diff --staged
git commit -m "docs: add Sprint 10 diff practice"
```

Meaning:

- you changed a file
 - checked the unstaged diff
 - staged the file
 - checked the staged diff
 - committed carefully
-

48. Practice B: Use Stash

```
echo "temporary work" >> temp.md  
git status  
git stash  
git status  
git stash list  
git stash pop
```

Meaning:

- you created unfinished work
 - stored it temporarily
 - checked the stash list
 - brought the work back
-

49. Practice C: Amend a Local Commit

```
echo "Forgotten line" >> README.md  
git add README.md  
git commit --amend
```

Meaning:

- you fixed the most recent local commit
 - you used amend instead of making a new extra cleanup commit
-

50. Practice D: Revert a Commit Safely

First inspect history:

```
git log
```

Then revert a practice commit:

```
git revert <commit-hash>
```

Meaning:

- you selected an old commit
 - you created a new undo commit safely
-

51. Practice E: Create a Tag

```
git tag v1.0.0  
git tag
```

Meaning:

- you marked the current commit as version `v1.0.0`
- you checked the list of tags

Optional richer version:

```
git tag -a v1.0.0 -m "release v1.0.0"
```

52. Practice F: Create a Workflow Folder and File

```
mkdir -p .github/workflows
```

Then create a file such as:

```
.github/workflows/simple-check.yml
```

Then commit it:

```
git add .github/workflows/simple-check.yml  
git commit -m "ci: add simple GitHub Actions check"  
git push
```

Meaning:

- you created the workflow folder
- you added a workflow file
- you committed CI configuration to the repository

Part 13: Command Summary

53. Core Sprint 10 Commands

Command	Simple Meaning
<code>git diff</code>	show unstaged changes
<code>git diff --staged</code>	show staged changes
<code>git restore <file></code>	discard local changes in a file
<code>git restore --staged <file></code>	unstage a file but keep edits
<code>git rm <file></code>	remove a file and stage that removal
<code>git mv oldname newname</code>	rename or move a file in Git work
<code>git stash</code>	temporarily save unfinished work
<code>git stash list</code>	show saved stashes
<code>git stash pop</code>	bring latest stash back and remove it
<code>git stash apply</code>	bring stash back but keep the stash entry
<code>git stash drop</code>	remove a stash entry
<code>git commit --amend</code>	fix the latest local commit
<code>git revert <commit></code>	create a safe undo commit
<code>git reset --soft HEAD~1</code>	undo last commit, keep staged changes
<code>git reset --mixed HEAD~1</code>	undo last commit, keep unstaged changes
<code>git reset --hard HEAD~1</code>	undo last commit and delete changes
<code>git reflog</code>	show recovery history
<code>git show <commit></code>	show one commit in detail
<code>git blame <file></code>	show who changed each line last
<code>git tag v1.0.0</code>	create a lightweight tag
<code>git tag -a v1.0.0 -m "release v1.0.0"</code>	create an annotated tag
<code>git push origin v1.0.0</code>	push a tag to GitHub
<code>git cherry-pick <commit></code>	copy one commit onto current branch
<code>git branch -r</code>	show remote branches

Command	Simple Meaning
<code>git fetch --all</code>	fetch updates from all remotes
<code>git pull --rebase</code>	pull and reapply local commits on top
<code>git switch -</code>	go back to previous branch
<code>git clean -n</code>	preview untracked file removal
<code>git clean -fd</code>	delete untracked files and folders
<code>git remote rename origin upstream</code>	rename a remote
<code>.github/workflows</code>	folder for GitHub Actions workflows

Part 14: Common Beginner Mistakes in Sprint 10

54. Mistake 1: Committing Without Checking the Diff

Bad habit:

- edit file
- stage file
- commit immediately

Better habit:

- run `git diff`
- stage file
- run `git diff --staged`
- commit

Simple meaning:

Check first, then commit.

55. Mistake 2: Using `git restore` Too Quickly

Problem: You remove changes you still needed.

Simple meaning:

Restore is useful, but it throws away local edits.

56. Mistake 3: Using `git commit --amend` on Shared Work

Problem: You rewrite a commit that other people may already know.

Simple meaning:

Amend is best for local unpushed cleanup.

57. Mistake 4: Using `git reset --hard` Carelessly

Problem: You may delete work permanently.

Simple meaning:

Hard reset is dangerous. Use it only when you clearly understand the risk.

58. Mistake 5: Using Reset When Revert Was Safer

Problem: You rewrite shared history instead of making a safe undo commit.

Simple meaning:

For shared commits, revert is usually safer than reset.

59. Mistake 6: Forgetting What Stash Does

Problem: You think stash is the same as a normal commit.

Correct idea:

- stash is temporary storage
- commit is normal project history

Simple meaning:

Stash hides unfinished work. Commit saves project history.

60. Mistake 7: Thinking CI/CD Means Only Deployment

Problem: You forget that CI is mostly about automated checking too.

Simple meaning:

CI checks code automatically. CD helps deliver or deploy it.

Part 15: Sprint 10 Checkpoint

61. Sprint 10 Checkpoint

You pass Sprint 10 when you can explain:

When should I use stash, revert, reset, and amend?

Good Beginner Answer

- `git stash` = when I have unfinished work but need to switch tasks or branches
- `git commit --amend` = when I need to fix my last local unpushed commit
- `git revert` = when I need to safely undo a commit that may already be shared
- `git reset --hard` = when I need to move local history backward, but I must be careful, especially with `--hard`

Simple final meaning:

Stash saves unfinished work temporarily. Amend fixes the latest local commit. Revert safely undoes shared commits. Reset rewrites local history and can be dangerous.

Part 16: Mini Assessment

62. Answer These Before Moving to Sprint 11

1. What does `git diff` show?
2. What does `git diff --staged` show?
3. What does `git restore <file>` do?
4. What does `git restore --staged <file>` do?
5. What is the difference between revert and reset?
6. When should you use `git stash`?
7. What does `git stash pop` do?
8. What is `git commit --amend` used for?
9. Why should you be careful with amend after pushing?
10. What does `git reflog` help you do?
11. What is a tag?
12. What is the difference between a lightweight tag and an annotated tag?
13. What does `git cherry-pick <commit>` do?

14. What is detached HEAD?
 15. What should you do if you want to keep work from detached HEAD?
 16. What does `git branch -r` show?
 17. What does `git fetch --all` do?
 18. What is branch protection?
 19. What is GitHub Flow?
 20. What is trunk-based development?
 21. What is Git Flow?
 22. What does CI mean?
 23. What does CD mean?
 24. What is GitHub Actions?
 25. Where do workflow files go?
 26. In GitHub Actions YAML, what do `jobs`, `steps`, `uses`, `with`, and `run` mean?
 27. Why is `git clean -n` safer than `git clean -fd`?
 28. What is `.gitattributes` useful for?
 29. What are conflict edge cases?
 30. Why is Sprint 10 important before moving to PostgreSQL and SQL?
-

Part 17: Completion Checklist

63. Tick These When Completed

- ☐ I can explain `git diff` in simple words.
- ☐ I can explain `git diff --staged` in simple words.
- ☐ I can safely use `git restore <file>`.
- ☐ I can unstage using `git restore --staged <file>`.
- ☐ I can explain what `git rm` and `git mv` do.
- ☐ I can explain what stash is and when to use it.
- ☐ I know the difference between `stash pop` and `stash apply`.
- ☐ I can explain `git commit --amend`.
- ☐ I can explain revert vs reset.
- ☐ I understand why `git reset --hard` is dangerous.
- ☐ I can explain what `git reflog` is used for.
- ☐ I can create and explain a Git tag.
- ☐ I can explain what `git cherry-pick` does.
- ☐ I understand detached HEAD.
- ☐ I can explain remote branches.
- ☐ I know what branch protection means.
- ☐ I can explain GitHub Flow.
- ☐ I can explain trunk-based development.
- ☐ I know Git Flow is a bigger branching model.
- ☐ I can explain CI/CD in simple words.
- ☐ I know that GitHub Actions uses YAML workflow files.
- ☐ I know that workflow files go inside `.github/workflows`.

- ☐ I can identify `jobs`, `steps`, `uses`, `with`, and `run` in a workflow file.
 - ☐ I completed the Sprint 10 checkpoint.
-

64. Final Simple Meaning

Sprint 10 teaches you the extra Git skills that make your work safer, cleaner, and more professional.

You are ready to move to Sprint 11 only after you can:

- inspect changes before committing
- undo the correct way
- use stash, amend, revert, and reset correctly
- understand tags, recovery, and branch safety
- explain CI/CD and the basic idea of GitHub Actions

Sprint 10 is the bridge between basic Git use and real team-level Git habits.